Industrie- und Handelskammer
für Ostfriesland und Papenburg

Bestätigung über die durchgeführte Projektarbeit im IT- Ausbildungsberuf Fachinformatiker für Anwendungsentwickl

Abschlussprüfung: Sommer 2026 Winter

Azubi-Identnummer: 121/ 572525

Ausbildungsfirma:

Name : Hamburg

Vorname: Niklas

Straße: Kapitän-Rüter-Str. 81

PLZ / Ort: 26871 Papenburg

Firma : journaway GmbH

Straße : Am alten Handelshafen 1

PLZ / Ort: 26789 Leer

Projektbezeichnung: Bereitstellung eines Microservices zur Synchronisation von dezentralisiert ges

Projektbeginn: 29.03.2026 Projektfertigstellung: 05.05.2026 Zeitaufwand in Std. 80

Bestätigung der Ausbildungsfirma:

Wir bestätigen, dass der Prüfling das oben bezeichnete Projekt einschließlich der Dokumentation im oben genannten Zeitraum selbständig ausgeführt hat.

Ort, Datum _____ Unterschrift Ausbildender

Ort, Datum _____ Unterschrift Projektverantwortliche/r

Eidesstattliche Erklärung des Prüflings:

Ich versichere, dass ich die Fachaufgabe selbständig durchgeführt und den dazugehörenden Report selbständig erstellt habe.

Papenburg, 05.05.2026 N.Hamburg

Ort, Datum Unterschrift Prüfling

Diese Bestätigung ist als 1. Seite der Dokumentation hochzuladen!

Abschlussprüfung Sommer 2026

Fachinformatiker für Anwendungsentwicklung

Dokumentation zur betrieblichen Projektarbeit

Bereitstellung eines Microservices

zur Synchronisation von dezentralisiert gespeicherten Kundendaten

Abgabetermin: Papenburg, den 08.05.2026

Prüfungsbewerber:
Niklas Hamburg
Kapitän-Rüter-Straße 81
26871 Papenburg

Ausbildungsbetrieb:
JOURNAWAY journaway GmbH
Am alten Handelshafen 1
26789 Leer

Inhaltsverzeichnis

Abbildungsverzeichnis **III**

Tabellenverzeichnis **IV**

Listings **V**

Abkürzungsverzeichnis **VI**

1 Einleitung **1**
1.1 Projektumfeld 1
1.2 Projektziel 1
1.3 Projektbegründung 1
1.4 Projektschnittstellen 2
1.5 Projektabgrenzung 2

2 Projektplanung **2**
2.1 Projektphasen 3
2.2 Abweichungen vom Projektantrag 3
2.3 Ressourcenplanung 3
2.4 Entwicklungsprozess 4

3 Analyse **4**
3.1 Ist-Analyse 4
3.2 Wirtschaftlichkeitsanalyse 5
3.3 Anforderungsanalyse 6
3.4 Nutzwertanalyse 7

4 Entwurf **7**
4.1 UML-Aktivitätsdiagramm 8
4.2 Architekturdesign 8
4.3 Datenbankstruktur 9

5 Implementierung **9**
5.1 Implementierung der Datenstrukturen 9
5.2 Implementierung der Benutzeroberfläche 10
5.3 Implementierung der Geschäftslogik 10

6 Abnahmephase **10**

7 Einführungsphase **11**

8 Dokumentation **11**

9 Fazit **11**
9.1 Soll-/Ist-Vergleich 11
9.2 Lessons Learned 12
9.3 Ausblick 12

Literaturverzeichnis **13**

Eidesstattliche Erklärung **14**

A Anhang **i**
A.1 Detaillierte Zeitplanung i
A.2 Synchronisations Abläufe ii
A.3 Datenbankmodell iii
A.4 Oberflächenentwürfe iv
A.5 Screenshots der Anwendung vi
A.6 Entwicklerdokumentation viii
A.7 Testfall und sein Aufruf auf der Konsole x
A.8 Benutzerdokumentation xi

Abbildungsverzeichnis

1 Nightcrawler ER-Diagramm iii
2 Liste der Module mit Filtermöglichkeiten iv
3 Anzeige der Übersichtsseite einzelner Module v
4 Anzeige und Filterung der Module nach Tags v
5 Anzeige und Filterung der Module nach Tags vi
6 Liste der Module mit Filtermöglichkeiten vii
7 Aufruf des Testfalls auf der Konsole xi

Tabellenverzeichnis

1 Zeitplanung 3
2 Soll-/Ist-Vergleich 12

Listings

1 Testfall in PHP x

Abkürzungsverzeichnis

API	Application Programming Interface
SQL	Structured Query Language
DLQ	Dead Letter Queue
TTL	Time-To-Live
ERD	Entity-Relationship-Diagramm

1 Einleitung

In dieser Dokumentation wird das Projekt 'Bereitstellung eines Microservices zur Synchronisation von dezentralisiert gespeicherten Kundendaten' beschrieben, welches im Rahmen der Prüfung zum Abschluss der Ausbildung zum Fachinformatiker Anwendungsentwicklung durchgeführt wurde. Es wird der Prozess der Entwicklung einer Anwendung dargestellt, die als eigenständiger Microservice implementiert wird.

1.1 Projektumfeld

JournawayGmbH ist ein in Leer ansässiges, 2017 gegründetes Reiseunternehmen und bietet Reisekomplett-Pakete auf ihrer Webseite an. Derzeit werden rund 160 Mitarbeiter beschäftigt und unter diesen Mitarbeitern arbeiten über ein Drittel in der internen IT-Abteilung, welche B2C und B2B Lösungen für das Unternehmen zur Verfügung stellt. Aufgaben dieser Abteilung liegen unteranderem darin die Webseite und die dazu gehörigen Daten, wie Buchungsdaten, Kundendaten etc. zu pflegen und zu ermöglichen, dass Preise und Informationen zu Leistungen innerhalb der Produkte stets aktuell sind.

Der Auftraggeber des Projektes ist die journaway GmbH selbst.

1.2 Projektziel

Journaway stellte in Auftrag ein Produkt zu entwickeln, welches als Schnittstelle Daten von zwei Kundendaten speichernden Systemen miteinander synchronisiert.

Mit der Entwicklung eines neuen Kundenportals auf der Webseite ist es notwendig, dass Informationen für die Kommunikation zwischen Unternehmen und Kunden nicht asynchron vorliegen. Teil dieser Aufgabe ist es einen Microservice zu entwickeln, der durch eventbasierte Synchronisierung die dezentrale Verfügbarkeit von Daten auflöst und eine einzige Source of Truth schafft, wodurch die Kundendaten auf beiden Systemen identisch sind.

1.3 Projektbegründung

In beiden bisherigen Systemen werden Kundendaten gespeichert, welche keine Relation zu einander haben und können somit für den gleichen Kunden unterschiedlich vorliegen. Mit dem neuen Microservice *Nightcrawler* sollen Fehler beim persistieren kundenkontaktrelevanter Informationen minimiert werden. Außerdem soll es ermöglichen das im bisherigen CRM *HubSpot* gespeicherte Kundendaten mit dem angemeldeten Kunden vom *user-service* verknüpft werden, wodurch im Kundenportal weitere, für den Kunden interessante Informationen dargestellt werden können.

Ohne den *Nightcrawler* müssten Mitarbeiter aus dem Kundensupport eigenhändig nach Abweichungen in Kontaktinformationen suchen und diese manuell einpflegen. Dies würde einen erhöhten Ressourcenaufwand und eine höhere Anfälligkeit für Fehler bedeuten. Ziel ist es, den Erhalt von zentralen und aktuellen Kontaktdaten, ohne manuellen Aufwand zu gewährleisten.

1.4 Projektschnittstellen

Die Anwendung interagiert über Event Queues durch RabbitMQ mit einem anderen Microservice *user-service*. Durch diesen Service ist es möglich, dass persönliche Informationen, Kommunikationspräferenzen oder Reisende von Websitebesuchern erstellt, bearbeitet und gelöscht werden können. Außerdem können die Nutzer dort auch ihre Buchungen einsehen.

Zudem kommuniziert die Anwendung über einen HttpClient mit dem CRM *HubSpot*. Dies ist der Ort an dem standardmäßig kundenbezogene Daten gespeichert werden. Dieser cloud-basierte Service von dem Unternehmen *HubSpot, Inc.* wird hauptsächlich für die Speicherung von Kontaktdaten, Deals und Ausführung von Workflows genutzt.

Dadurch, dass es sich um einen autonomen Synchronisationsservice handelt gibt es keine direkten Benutzer, jedoch werden events zur Synchronisierung entweder durch den Websitenutzer oder Mitarbeiter in *HubSpot* ausgelöst. Schließlich werden die Synchronisationshistorien zu Kunden in einer PostgreSql Datenbank persistiert.

Der *user-service*, als auch der *Nightcrawler* sind Microservices aufbauend auf dem .NET 10 Framework von Microsoft und werden auf Servern mit Linux basiertem Betriebssystem (Debian 12.13) ausgeführt. Der Quellcode ist in der Sprache CSharp geschrieben. Als Entwicklungsumgebung wird JetBrains Rider 2026 verwendet.

1.5 Projektabgrenzung

Teil dieses Projektes ist die Planung und Entwicklung eines funktionellen und testbaren Prototyps. Die Aufnahme der Anwendung in den Livebetrieb wird nicht behandelt.

2 Projektplanung

Um einen möglichst reibungslosen Ablauf in der Entwicklung des Microservices zu gewährleisten, braucht es eine gute Planung. In diesem Abschnitt wird daher aufgezeigt, welche Ressourcen (Zeit, Hardware) wie eingesetzt werden.

2.1 Projektphasen

Die Umsetzung des Projektes findet während zwei vier Wochen Sprints statt und wird gemäß der Vorgaben der IHK, wie in Tabelle 1 dargestellt, innerhalb von 80 Stunden bearbeitet:

Zeitplanung Tabelle 1 zeigt die grobe Zeitplanung.

Projektphase	Geplante Zeit
Analyse	8 h
Entwurf	6 h
Implementierung	49 h
Einführung	7 h
Dokumentation	10 h
Gesamt	**80 h**

Tabelle 1: Zeitplanung

Eine detailliertere Zeitplanung findet sich im Anhang A.1: Detaillierte Zeitplanung auf Seite i.

2.2 Abweichungen vom Projektantrag

Bei dem Inhalt des Projektes wird die Anbindung von Hangfire mit der Anbindung von RabbitMQ ausgetauscht, da bei der Planung erkannt wurde, dass Hangfire in der ersten Iteration nicht benötigt wird und sich bei der Umsetzung der Event Queues auf den Message Broker RabbitMQ geeinigt wurde.

2.3 Ressourcenplanung

Hardware

1. Büroarbeitsplatz
2. Netzwerk mit Internetzugang
3. Laptop und zwei weitere Bildschirme
4. Server für die Anwendung und Datenbank

Software

1. Betriebssystem Laptop: Windows 11
2. Entwicklungsumgebung: JetBrains Rider 2026
3. Message Broker Software: RabbitMQ

4. Datenbanksoftware: PostgreSQL 16
5. Zielbetriebssystem: Linux Debian 12.13
6. Software zum generieren und einsehen von Testdaten auf der Datenbank: HeidiSQL

Personal

1. Für die Qualitätssicherung während der Review- und Testphase werden zwei weitere Entwickler benötigt.

2.4 Entwicklungsprozess

Während der Umsetzung des Projektes wird ein agiler Prozess bzw. Scrum verfolgt. Dies ist ein Prozess, bei dem iterativ kleinere Abschnitte eines Auftrages fertiggestellt werden, wodurch schnell auf unvorhergesehene Probleme oder neue Anforderungen eingegangen werden kann. Scrum zeichnet sich durch eine hohe Flexibilität aus und eignet sich gut für kleine Projekte bzw. Teams.

3 Analyse

Im Folgenden wird zunächst die bisherige Situation im Unternehmen betrachtet. Darauf aufbauend wird untersucht, welchen wirtschaftlichen Nutzen das Projekt bietet. Anschließend werden die fachlichen und technischen Anforderungen an die Anwendung beschrieben. Zum Abschluss folgt eine Nutzwertanalyse, um den nicht-monetären Nutzen des Projektes zu bewerten.

3.1 Ist-Analyse

Aktuell werden kundenbezogene Daten in zwei voneinander getrennten Systemen gespeichert. Zum einen im *user-service*, welcher Daten für das Kundenportal verwaltet, und zum anderen im CRM-System *HubSpot*. Zwischen beiden Systemen besteht bislang keine eigenständig verwaltete und durchgängig nachvollziehbare Synchronisation. Dadurch kann es passieren, dass Kontaktinformationen eines Kunden in beiden Systemen unterschiedlich vorliegen.

Besonders problematisch ist dies bei Informationen, die für die Kommunikation mit dem Kunden relevant sind. Dazu zählen unter anderem die E-Mail-Adresse, der Vorname, der Nachname, die Anrede sowie Kommunikationspräferenzen. Werden diese Daten in einem der beiden Systeme geändert, ist nicht sichergestellt, dass die Änderung zuverlässig im jeweils anderen System übernommen wird.

Zusätzlich existiert bislang keine zentrale Stelle, die nachvollziehbar speichert, wann eine Synchronisation durchgeführt wurde, aus welchem System eine Änderung stammt und ob diese erfolgreich

verarbeitet wurde. Fehler bei der Übertragung könnten dadurch nur mit erhöhtem manuellem Aufwand erkannt und korrigiert werden. Ebenso gibt es bisher keine eigenständige technische Lösung, um mit temporären Fehlern, Systemausfällen oder Rate Limits der HubSpot API umzugehen.

Aus diesem Grund soll mit dem Projekt *Nightcrawler* ein eigenständiger Microservice entwickelt werden, der die Synchronisation zwischen beiden Systemen übernimmt. Hierzu soll eine PostgreSQL-Datenbank eingeführt werden, in der sowohl der aktuelle Synchronisationszustand zwischen Benutzer und Kontakt, als auch die Historie aller Synchronisationsereignisse gespeichert wird.

Außerdem soll die Kommunikation zwischen den beteiligten Systemen asynchron über RabbitMQ erfolgen. Dafür werden acht Event Queues benötigt: eine Queue für Änderungen vom *user-service* in Richtung *HubSpot*, eine Queue für Änderungen von *HubSpot* in Richtung *user-service* sowie sechs weitere Queues, die jeweils pro Richtung für zwei erneute Versuche und einen Fehlschlag zuständig sind.

Die bisherige Situation soll also dahingehend verbessert werden, dass Kundendaten zwischen beiden Systemen automatisch, nachvollziehbar und fehlertolerant synchronisiert werden können. Zusätzlich soll die neue Anwendung in der Lage sein, Konflikte bei gleichzeitigen Änderungen zu erkennen und nach festgelegten Regeln aufzulösen.

3.2 Wirtschaftlichkeitsanalyse

Die Entwicklung des Microservices verursacht zunächst einmalige Projektkosten in Form von Arbeitszeit. Für die Umsetzung des Projektes sind insgesamt 80 Stunden vorgesehen. Bei einem internen Verrechnungssatz von 35 Euro Netto pro Stunde ergeben sich folgende Entwicklungskosten:

$$80 \text{ Stunden} \times 35 \text{ Euro/Stunde} = 2800 \text{ Euro}$$

Dem gegenüber steht der Nutzen, der sich aus der automatisierten Synchronisation von Kundendaten ergibt. Ohne eine solche Lösung müssten Abweichungen zwischen den Datenbeständen manuell erkannt, überprüft und korrigiert werden. Dies würde insbesondere im Kundensupport und in der technischen Betreuung zusätzlichen Aufwand verursachen.

Zusätzlich sinkt durch die Anwendung das Risiko, dass veraltete oder fehlerhafte Kontaktdaten in einem der Systeme verwendet werden. Dadurch können Fehler in Folgeprozessen reduziert werden, beispielsweise bei der Kundenkommunikation oder bei der Darstellung von Informationen im Kundenportal.

Wird angenommen, dass durch die automatisierte Synchronisation wöchentlich 2,5 Stunden manueller Aufwand eingespart werden können, ergibt sich bei einem internen Stundensatz von 15,50 Euro Netto pro Stunde ein jährliches Einsparpotenzial von:

$$2{,}5 \text{ Stunden/Woche} \times 15{,}50 \text{ Euro/Stunde} \times 52 \text{ Wochen} = 2015 \text{ Euro/Jahr}$$

Neben dieser direkten Einsparung entsteht zusätzlicher Nutzen durch eine höhere Datenqualität, bessere Nachvollziehbarkeit und eine geringere Fehleranfälligkeit. Das Projekt ist damit wirtschaftlich sinnvoll, da die Entwicklungskosten mittelfristig durch eingesparte Arbeitszeit und verbesserte Prozesse ausgeglichen werden können.

3.3 Anforderungsanalyse

Damit das Projektziel erreicht werden kann, muss die Anwendung sowohl fachliche als auch technische Anforderungen erfüllen.

Fachlich muss der *Nightcrawler* in der Lage sein, Änderungen an Kundendaten zwischen dem *user-service* und *HubSpot* in beide Richtungen zu synchronisieren. Dabei sollen mindestens die Datenfelder *email*, *givenName*, *surname*, *salutation* und *CommunicationPreference* berücksichtigt werden.

Wenn ein Benutzer im *user-service* erstellt oder geändert wird, soll ein entsprechendes Ereignis verarbeitet und die dazugehörigen Daten in *HubSpot* aktualisiert werden. Wird ein Benutzer neu erstellt, soll zunächst geprüft werden, ob anhand der E-Mail-Adresse bereits ein passender Kontakt in *HubSpot* existiert. Ist dies der Fall, soll dieser Kontakt aktualisiert werden. Falls kein Kontakt gefunden wird, soll ein neuer Kontakt erstellt werden.

Wenn dagegen in *HubSpot* ein Kontakt geändert wird, soll diese Änderung ebenfalls verarbeitet und an den *user-service* weitergegeben werden. Kontakte aus *HubSpot*, für die kein passender Benutzer im *user-service* existiert, sollen zunächst ignoriert werden.

Damit jede Synchronisation nachvollziehbar bleibt, muss jedes verarbeitete Ereignis zunächst in einer Historie gespeichert werden. Diese *SyncHistory* soll Informationen zur Quelle des Ereignisses, zum Ereignistyp, zu Benutzer- und Kontaktzuordnung, zum Payload, zum Zeitpunkt des Auftretens, zum aktuellen Status sowie zur Anzahl der bisherigen Verarbeitungsversuche enthalten.

Zusätzlich muss ein *UserContactState* gespeichert werden. Dieser dient dazu, die Zuordnung zwischen Benutzer und HubSpot-Kontakt sowie den letzten bekannten Synchronisationsstand zu verwalten. Dabei sollen unter anderem die Zeitpunkte der letzten Änderung im *user-service*, der letzten Änderung in *HubSpot* und der letzten erfolgreichen Synchronisation gespeichert werden.

Technisch muss die Anwendung acht Event Queues über RabbitMQ verwenden. Da keine Nachrichten bei einem Neustart oder Ausfall verloren gehen sollen, müssen die Hauptqueues als durable Quorum Queues umgesetzt werden. Für temporäre Fehler soll eine Retry-Logik mit TTL-basierten Retry Queues eingesetzt werden. Nicht behandelbare Fehler sollen in einer Dead-Letter-Queue abgelegt werden.

Zur Konfliktbehandlung soll eine optimistische Strategie verwendet werden. Wenn Daten in beiden Systemen unterschiedlich geändert wurden, soll der Datenstand des *user-service* bevorzugt werden. Wird also eine Änderung aus *HubSpot* erkannt, die im Konflikt mit aktuelleren Daten aus dem *user-service* steht, soll diese Änderung nicht übernommen, sondern stattdessen eine neue Synchronisation in Richtung *HubSpot* angestoßen werden.

3.4 Nutzwertanalyse

Neben dem wirtschaftlichen Nutzen bietet das Projekt auch einen deutlichen nicht-monetären Mehrwert. Insbesondere die Datenqualität, die Nachvollziehbarkeit von Änderungen und die technische Stabilität der Synchronisation werden verbessert.

Vor der Umsetzung des Projektes besteht das Problem, dass Änderungen an Kundendaten nicht zentral nachvollzogen werden können. Außerdem ist die Synchronisation zwischen beiden Systemen nicht eigenständig abgesichert, wodurch Fehler bei der Übertragung schwerer zu erkennen sind. Mit dem *Nightcrawler* wird eine Lösung geschaffen, die Änderungen dokumentiert, Synchronisationszustände speichert und Fehlerfälle gezielt behandeln kann.

Zur Bewertung des Nutzwertes werden die Kriterien Datenkonsistenz, Nachvollziehbarkeit, Ausfallsicherheit, Wartbarkeit und Automatisierungsgrad betrachtet. Die folgende Matrix stellt einen Vorher-/Nachher-Vergleich dar:

Kriterium	Gewichtung	Vorher	Nachher
Datenkonsistenz	5	2	5
Nachvollziehbarkeit	3	1	5
Ausfallsicherheit	3	2	4
Wartbarkeit	2	2	4
Automatisierungsgrad	2	2	4
Gesamt:	**15**	**27**	**68**
Nutzwert:		**1,8**	**4,53**

Aus den Bewertungen ergibt sich, dass der Nutzen der neuen Anwendung vor allem in der zuverlässigeren Verarbeitung, der besseren Transparenz und der geringeren manuellen Abhängigkeit liegt.

Das Projekt verbessert somit nicht nur die technische Grundlage für die Synchronisation von Kundendaten, sondern schafft auch eine stabilere und besser wartbare Lösung für zukünftige Erweiterungen.

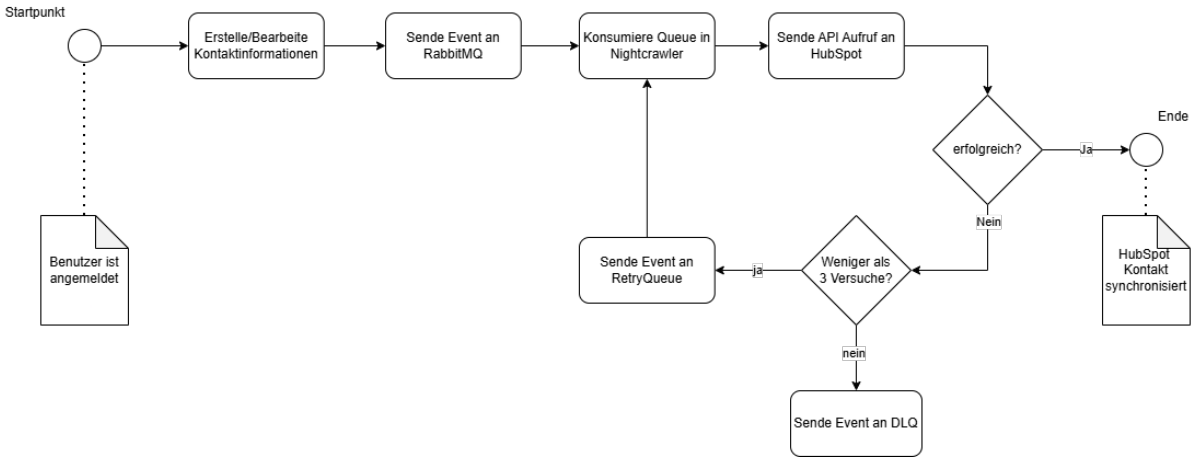
4 Entwurf

In dieser Phase soll die Implementierung vorbereitet werden. Dazu wird in einem UML-Aktivitätsdiagramm dar- gestellt, wie der Synchronisationsablauf in eine Richtung aussieht. Anschließend zeigt eine Archi-

tekturübersicht, wie diese Systeme miteinander integagieren. Am Ende stellt ein ER-Modell dar, wie die Datenbankstruktur für die Event Persistierung aussieht.

4.1 UML-Aktivitätsdiagramm

Das folgende UML-Aktivitätsdiagramm für die Synchronisation in Richtung *HubSpot*, zeigt den Ablauf der Synchronisation vom *user-service* ausgehend. Angefangen mit der Eingabe der Kontaktdaten vom Nutzer wird ein Event in die Main Queue geschickt. Anschließend konsumiert der *Nightcrawler* diese Queue und sendet basierend auf den Daten innerhalb der Event Message, einen API Upsert Aufruf an *HubSpot*. Sofern der API Aufruf einen 201 Created oder 200 Ok Statuscode zurückliefert wird das Event als erfolgreich angesehen. Andernfalls wird das Event in eine Retry Queue geschickt und nach einer 30 sekündigen TTL beim ersten Mal und 60 Sekunden beim zweiten Mal erneut ausgeführt. Wenn das zweite Mal ebenfalls nicht erfolgreich ist, wird das Event in eine DLQ geschickt.



Für genauere technische Abläufe siehe Anhang A.2: Synchronisations Abläufe auf Seite ii

4.2 Architekturdesign

Bei der ausgwählten Architektur handelt es sich um eine eine Orchestrator-Architektur, die es ermöglicht die Steuerung komplexer Arbeitsabläufe in Microservices zentralisiert auszuführen. Es gibt also ein Orchestrator Project, welches das Wissen über Konfigurationen beinhaltet, um andere Services zu registrieren und aufzurufen. Durch die Zentralisierung des Kontrollablaufs und die Entscheidungsgewalt über die nächsten Programmaufrufe, erleichtert der Orchestrator die Koordination verschiedener Services.

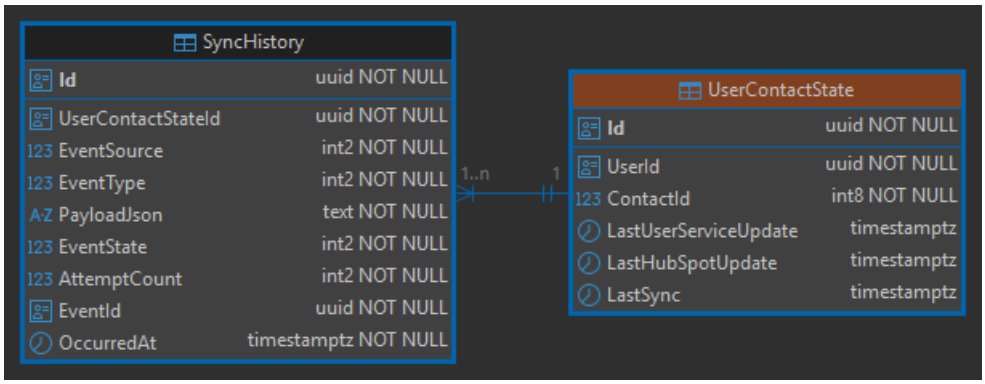
Die Entwicklung der Anwendung findet innerhalb des Microsoft .NET 10 Frameworks statt. .NET 10, veröffentlicht am 11. November 2025, ist ein plattformunabhängiges, modulares Framework, das die Entwicklung für Desktop, Web, Mobile, Cloud und IoT vereinfacht. Es basiert auf dem SDK-Modell,

das eine gleichzeitige Entwicklung und Ausführung auf Windows, Linux und macOS ermöglicht, und integriert die Laufzeitkomponenten CoreCLR, CoreFX sowie ASP.NET Core. Die Funktionsweise wird durch CSharp 14 geprägt, das Sprachfeatures wie Primary Constructors für alle Typen, Params-Spans für die Speicheroptimierung und Lambda-Verbesserungen bietet. Ein zentraler Meilenstein ist die Dateibasierte App-Unterstützung, die es erlaubt, einzelne *.cs-Dateien direkt auszuführen (dotnet run main.cs), ohne eine traditionelle Projektdatei zu benötigen. Dieses Framework wird regelmäßig erneuert, um den heutigen Standards gerecht zu werden. Außerdem bietet .NET eine große Auswahl an Funktionen und ermöglicht mithilfe von weiteren NuGet Paketen, wie zum Beispiel Entity Framework Core, eine vereinfachte Arbeit mit Entitäten aus Listen, Datenbanken und mehr.

4.3 Datenbankstruktur

Die Datenstruktur liegt zur Veranschaulichung im Folgenden als ERD vor. Sie besteht aus zwei Tabellen *SyncHistory* und *UserContactState*, welche in einer 1:n Beziehung zu einander stehen. Der *UserContactState* beschreibt einen Nutzer, welcher mit *HubSpot* verknüpft wird und die *SyncHistory* beschreibt alle Aktivitäten, die dieser Nutzer im Bezug auf dessen Daten gemacht hat.

Nightcrawler Datenbank Hier wird ein Entity-Relationship-Diagramm (ERD) dargestellt, welches Attribute, Relationen und die dazugehörigen Multiplizitäten enthält.



Anhang A.3: Datenbankmodell auf Seite iii

5 Implementierung

5.1 Implementierung der Datenstrukturen

- Beschreibung der angelegten Datenbank (z. B. Generierung von SQL aus Modellierungswerkzeug oder händisches Anlegen), **XML!**-Schemas usw..

5.2 Implementierung der Benutzeroberfläche

- Beschreibung der Implementierung der Benutzeroberfläche, falls dies separat zur Implementierung der Geschäftslogik erfolgt (z. B. bei **HTML!**-Oberflächen und Stylesheets).
- Ggfs. Beschreibung des Corporate Designs und dessen Umsetzung in der Anwendung.
- Screenshots der Anwendung

Beispiel Screenshots der Anwendung in der Entwicklungsphase mit Dummy-Daten befinden sich im Anhang A.5: Screenshots der Anwendung auf Seite vi.

5.3 Implementierung der Geschäftslogik

- Beschreibung des Vorgehens bei der Umsetzung/Programmierung der entworfenen Anwendung.
- Ggfs. interessante Funktionen/Algorithmen im Detail vorstellen, verwendete Entwurfsmuster zeigen.
- Quelltextbeispiele zeigen.
- Hinweis: Wie in Kapitel 1: Einleitung zitiert, wird nicht ein lauffähiges Programm bewertet, sondern die Projektdurchführung. Dennoch würde ich immer Quelltextausschnitte zeigen, da sonst Zweifel an der tatsächlichen Leistung des Prüflings aufkommen können.

Beispiel Die Klasse `ComparedNaturalModuleInformation` findet sich im Anhang **??**: **??** auf Seite ??.

6 Abnahmephase

- Welche Tests (z. B. Unit-, Integrations-, Systemtests) wurden durchgeführt und welche Ergebnisse haben sie geliefert (z. B. Logs von Unit Tests, Testprotokolle der Anwender)?
- Wurde die Anwendung offiziell abgenommen?

Beispiel Ein Auszug eines Unit Tests befindet sich im Anhang A.7: Testfall und sein Aufruf auf der Konsole auf Seite x. Dort ist auch der Aufruf des Tests auf der Konsole des Webservers zu sehen.

7 Einführungsphase

- Welche Schritte waren zum Deployment der Anwendung nötig und wie wurden sie durchgeführt (automatisiert/manuell)?
- Wurden ggfs. Altdaten migriert und wenn ja, wie?
- Wurden Benutzerschulungen durchgeführt und wenn ja, Wie wurden sie vorbereitet?

8 Dokumentation

- Wie wurde die Anwendung für die Benutzer/Administratoren/Entwickler dokumentiert (z. B. Benutzerhandbuch, API-Dokumentation)?
- Hinweis: Je nach Zielgruppe gelten bestimmte Anforderungen für die Dokumentation (z. B. keine IT-Fachbegriffe in einer Anwenderdokumentation verwenden, aber auf jeden Fall in einer Dokumentation für den IT-Bereich).

Beispiel Ein Ausschnitt aus der erstellten Benutzerdokumentation befindet sich im Anhang A.8: Benutzerdokumentation auf Seite xi. Die Entwicklerdokumentation wurde mittels PHPDoc[1] automatisch generiert. Ein beispielhafter Auszug aus der Dokumentation einer Klasse findet sich im Anhang A.6: Entwicklerdokumentation auf Seite viii.

9 Fazit

9.1 Soll-/Ist-Vergleich

- Wurde das Projektziel erreicht und wenn nein, warum nicht?
- Ist der Auftraggeber mit dem Projektergebnis zufrieden und wenn nein, warum nicht?
- Wurde die Projektplanung (Zeit, Kosten, Personal, Sachmittel) eingehalten oder haben sich Abweichungen ergeben und wenn ja, warum?
- Hinweis: Die Projektplanung muss nicht strikt eingehalten werden. Vielmehr sind Abweichungen sogar als normal anzusehen. Sie müssen nur vernünftig begründet werden (z. B. durch Änderungen an den Anforderungen, unter-/überschätzter Aufwand).

Beispiel (verkürzt) Wie in Tabelle 2 zu erkennen ist, konnte die Zeitplanung bis auf wenige Ausnahmen eingehalten werden.

[1] Vgl. **?**

Phase	Geplant	Tatsächlich	Differenz
Analyse	19 h	19 h	
Entwurf	9 h	10 h	+1 h
Implementierung	29 h	28 h	-1 h
Einführung	1 h	1 h	
Dokumentation	9 h	11 h	+2 h
Gesamt	70 h	70 h	

Tabelle 2: Soll-/Ist-Vergleich

9.2 Lessons Learned

- Was hat der Prüfling bei der Durchführung des Projekts gelernt (z. B. Zeitplanung, Vorteile der eingesetzten Frameworks, Änderungen der Anforderungen)?

9.3 Ausblick

- Wie wird sich das Projekt in Zukunft weiterentwickeln (z. B. geplante Erweiterungen)?

Literaturverzeichnis

Eidesstattliche Erklärung

Ich, Niklas Hamburg, versichere hiermit, dass ich meine **Dokumentation zur betrieblichen Projektarbeit** mit dem Thema

> *Bereitstellung eines Microservices – zur Synchronisation von dezentralisiert gespeicherten Kundendaten*

selbständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe, wobei ich alle wörtlichen und sinngemäßen Zitate als solche gekennzeichnet habe. Die Arbeit wurde bisher keiner anderen Prüfungsbehörde vorgelegt und auch nicht veröffentlicht.

Papenburg, den 08.05.2026

Niklas Hamburg

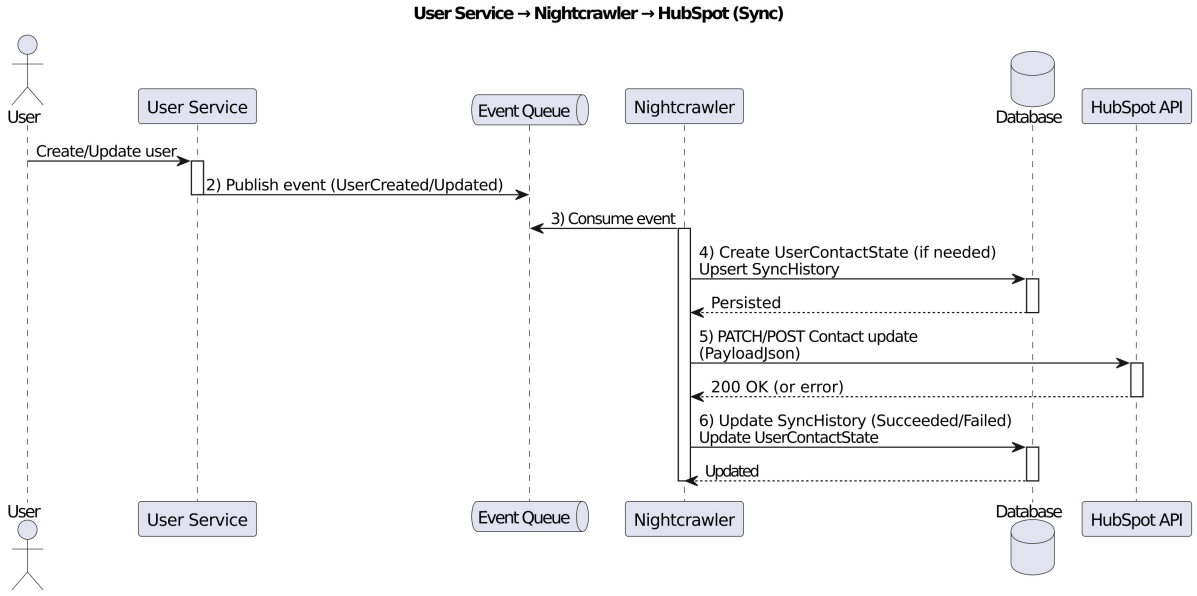
A Anhang

A.1 Detaillierte Zeitplanung

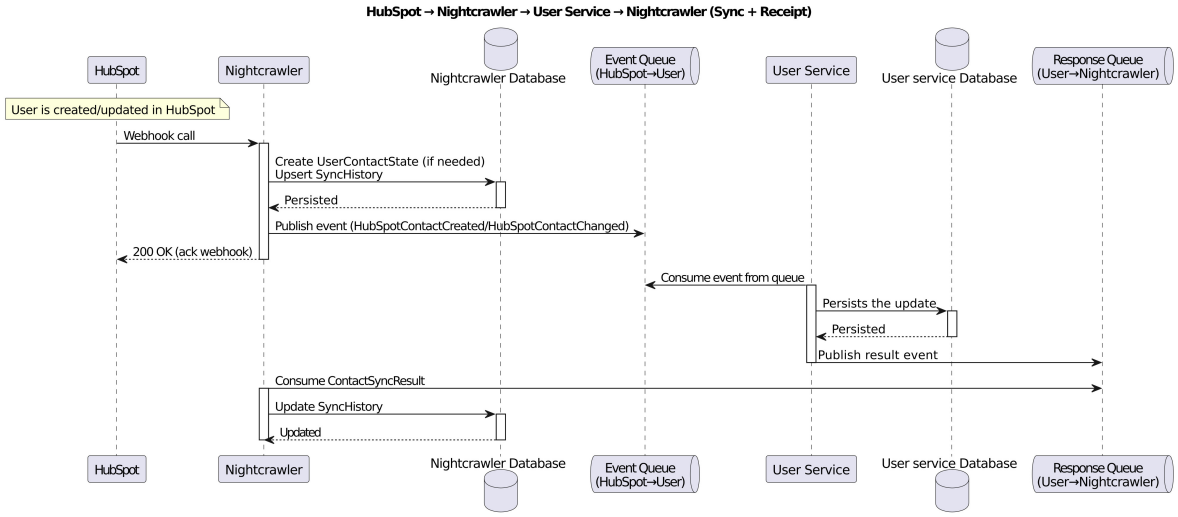
Analysephase			**9 h**
1. Analyse des Ist-Zustands		3 h	
1.1. Fachgespräch mit der EDV-Abteilung	1 h		
1.2. Prozessanalyse	2 h		
2. „Make or buy“-Entscheidung und Wirtschaftlichkeitsanalyse		1 h	
3. Erstellen eines „Use-Case“-Diagramms		2 h	
4. Erstellen des Lastenhefts mit der EDV-Abteilung		3 h	
Entwurfsphase			**19 h**
1. Prozessentwurf		2 h	
2. Datenbankentwurf		3 h	
2.1. ER-Modell erstellen	2 h		
2.2. Konkretes Tabellenmodell erstellen	1 h		
3. Erstellen von Datenverarbeitungskonzepten		4 h	
3.1. Verarbeitung der CSV-Daten	1 h		
3.2. Verarbeitung der SVN-Daten	1 h		
3.3. Verarbeitung der Sourcen der Programme	2 h		
4. Benutzeroberflächen entwerfen und abstimmen		2 h	
5. Erstellen eines UML-Komponentendiagramms der Anwendung		4 h	
6. Erstellen des Pflichtenhefts		4 h	
Implementierungsphase			**29 h**
1. Anlegen der Datenbank		1 h	
2. Umsetzung der HTML-Oberflächen und Stylesheets		4 h	
3. Programmierung der PHP-Module für die Funktionen		23 h	
3.1. Import der Modulinformationen aus CSV-Dateien	2 h		
3.2. Parsen der Modulquelltexte	3 h		
3.3. Import der SVN-Daten	2 h		
3.4. Vergleichen zweier Umgebungen	4 h		
3.5. Abrufen der von einem zu wählenden Benutzer geänderten Module	3 h		
3.6. Erstellen einer Liste der Module unter unterschiedlichen Aspekten	5 h		
3.7. Anzeigen einer Liste mit den Modulen und geparsten Metadaten	3 h		
3.8. Erstellen einer Übersichtsseite für ein einzelnes Modul	1 h		
4. Nächtlichen Batchjob einrichten		1 h	
Abnahmetest der Fachabteilung			**1 h**
1. Abnahmetest der Fachabteilung		1 h	
Einführungsphase			**1 h**
1. Einführung/Benutzerschulung		1 h	
Erstellen der Dokumentation			**9 h**
1. Erstellen der Benutzerdokumentation		2 h	
2. Erstellen der Projektdokumentation		6 h	
3. Programmdokumentation		1 h	
3.1. Generierung durch PHPdoc	1 h		
Pufferzeit			**2 h**
1. Puffer		2 h	
Gesamt			**70 h**

A.2 Synchronisations Abläufe

User-Service zu HubSpot



HubSpot zu User-Service



A.3 Datenbankmodell

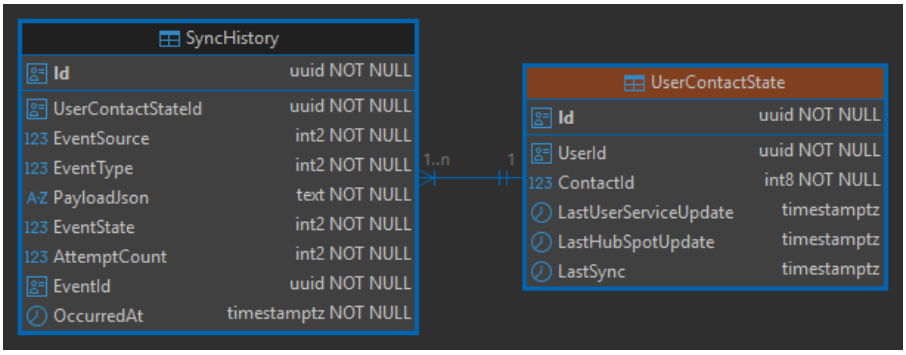


Abbildung 1: Nightcrawler ER-Diagramm

A.4 Oberflächenentwürfe

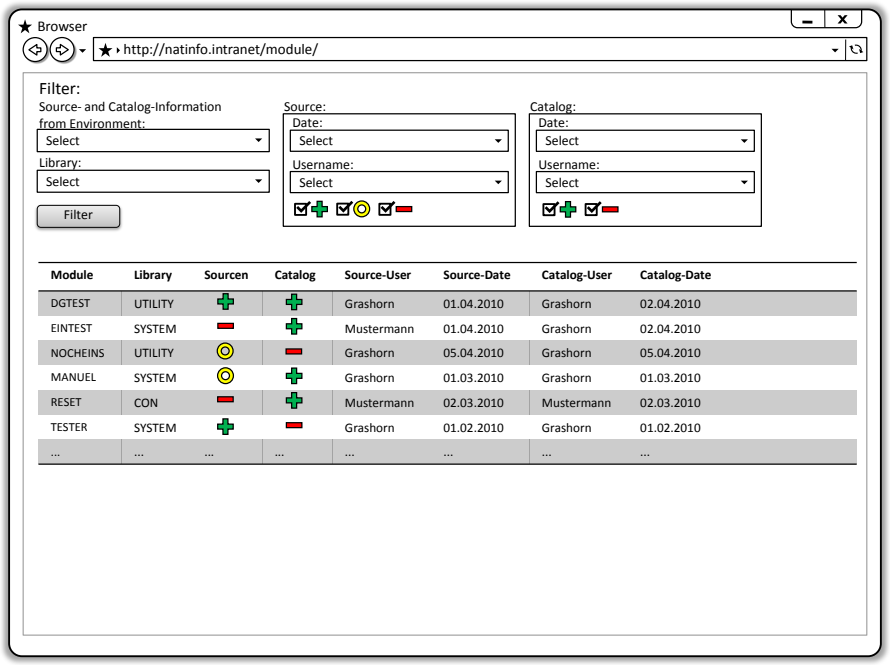


Abbildung 2: Liste der Module mit Filtermöglichkeiten

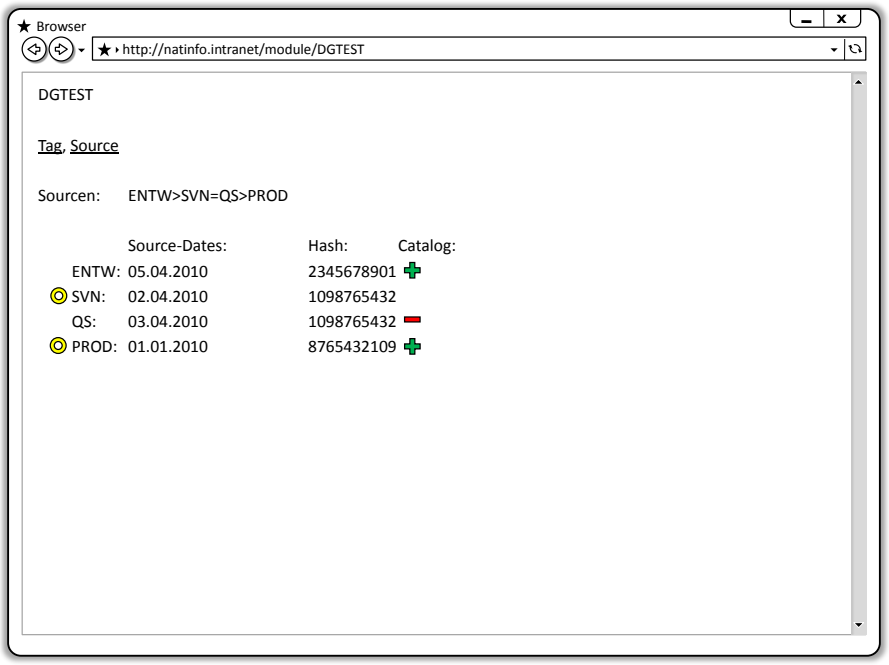


Abbildung 3: Anzeige der Übersichtsseite einzelner Module

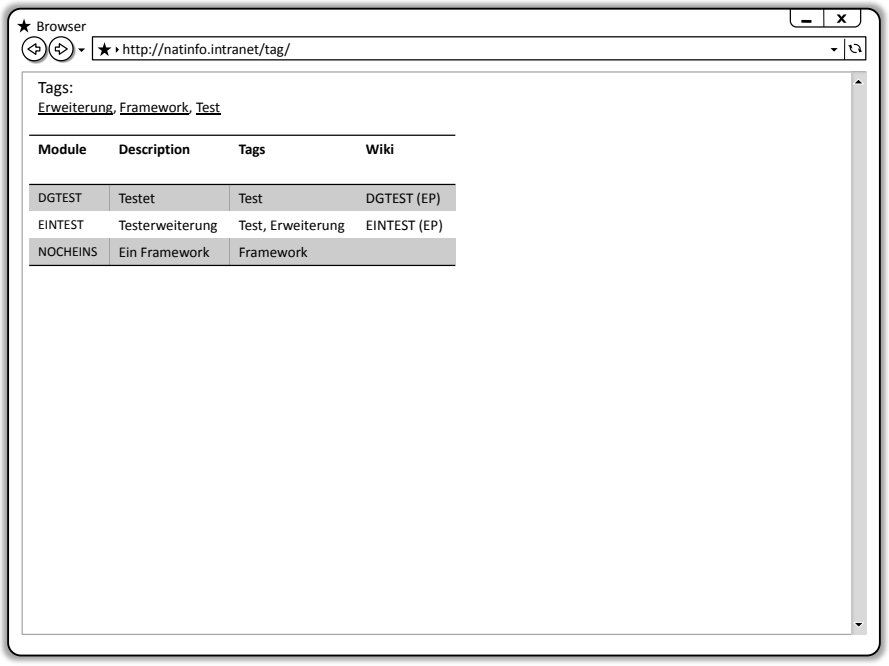


Abbildung 4: Anzeige und Filterung der Module nach Tags

A.5 Screenshots der Anwendung

Tags

Project, Test

Modulename	Description	Tags	Wiki
DGTEST	Macht einen ganz tollen Tab.	HGP	SMTAB_(EP), b
MALWAS		HGP, Test	
HDRGE		HGP, Project	
WURAM		HGP, Test	
PAMIU		HGP	

Abbildung 5: Anzeige und Filterung der Module nach Tags

Modules

Environment	ENTW
Library	Select
Catalog user	Select
Catalog date	Select
Source user	Select
Source date	Select

Reset Filter

Name	Library	Source	Catalog	Source-User	Source-Date	Catalog-User	Catalog-Date
SMTAB	UTILITY			MACKE	01.04.2010 13:00	MACKE	01.04.2010 13:00
DGTAB	CON			GRASHORN	01.04.2010 13:00	GRASHORN	01.04.2010 13:00
DGTEST	SUP			GRASHORN	05.04.2010 13:00	GRASHORN	05.04.2010 13:00
OHNETAG	CON			GRASHORN	05.04.2010 13:00	GRASHORN	01.04.2010 15:12
OHNEWIKI	CON			GRASHORN	05.04.2010 13:00	MACKE	01.04.2010 15:12

Abbildung 6: Liste der Module mit Filtermöglichkeiten

A.6 Entwicklerdokumentation

lib-model

[class tree: lib-model] [index: lib-model] [all elements]

Packages:
lib-model

Files:
Naturalmodulename.php

Classes:
Naturalmodulename

Class: Naturalmodulename

Source Location: /Naturalmodulename.php

Class Overview

```
BaseNaturalmodulename
    |
    --Naturalmodulename
```

Subclass for representing a row from the 'NaturalModulename' table.

Methods

- __construct
- getNaturalTags
- getNaturalWikis
- loadNaturalModuleInformation
- __toString

Class Details

[line 10]
Subclass for representing a row from the 'NaturalModulename' table.

Adds some business logic to the base.

[Top]

Class Methods

constructor __construct [line 56]

```
Naturalmodulename __construct( )
```

Initializes internal state of Naturalmodulename object.

Tags:

see: parent::__construct()
access: public

[Top]

method getNaturalTags [line 68]

```
array getNaturalTags( )
```

Returns an Array of NaturalTags connected with this Modulename.

Tags:

return: Array of NaturalTags
access: public

[Top]

method getNaturalWikis [line 83]

```
array getNaturalWikis( )
```

Returns an Array of NaturalWikis connected with this Modulename.

Tags:

return: Array of NaturalWikis
access: public

[Top]

method loadNaturalModuleInformation [line 17]

```
ComparedNaturalModuleInformation
loadNaturalModuleInformation( )
```

Gets the ComparedNaturalModuleInformation for this NaturalModulename.

Tags:

access: public

[Top]

method __toString [line 47]

```
string __toString( )
```

Returns the name of this NaturalModulename.

Tags:

access: public

[Top]

Documentation generated on Thu, 22 Apr 2010 08:14:01 +0200 by phpDocumentor 1.4.2

A.7 Testfall und sein Aufruf auf der Konsole

```
<?php
include(dirname(__FILE__).'/../bootstrap/Propel.php');

$t = new lime_test(13);

$t->comment('Empty Information');
$emptyComparedInformation = new ComparedNaturalModuleInformation(array());
$t->is($emptyComparedInformation->getCatalogSign(), ComparedNaturalModuleInformation::EMPTY_SIGN, '
    Has no catalog sign');
$t->is($emptyComparedInformation->getSourceSign(), ComparedNaturalModuleInformation::SIGN_CREATE, '
    Source has to be created');

$t->comment('Perfect Module');
 $criteria = new Criteria();
$criteria->add(NaturalmodulenamePeer::NAME, 'SMTAB');
$moduleName = NaturalmodulenamePeer::doSelectOne($criteria);
$t->is($moduleName->getName(), 'SMTAB', 'Right modulename selected');
$comparedInformation = $moduleName->loadNaturalModuleInformation();
$t->is($comparedInformation->getSourceSign(), ComparedNaturalModuleInformation::SIGN_OK, 'Source sign
    shines global');
$t->is($comparedInformation->getCatalogSign(), ComparedNaturalModuleInformation::SIGN_OK, 'Catalog sign
    shines global');
$infos = $comparedInformation->getNaturalModuleInformations();
foreach($infos as $info)
{
  $env = $info->getEnvironmentName();
  $t->is($info->getSourceSign(), ComparedNaturalModuleInformation::SIGN_OK, 'Source sign shines at ' . $env);
  if($env != 'SVNENTW')
  {
    $t->is($info->getCatalogSign(), ComparedNaturalModuleInformation::SIGN_OK, 'Catalog sign shines at ' .
        $info->getEnvironmentName());
  }
  else
  {
    $t->is($info->getCatalogSign(), ComparedNaturalModuleInformation::EMPTY_SIGN, 'Catalog sign is empty
        at ' . $info->getEnvironmentName());
  }
}
?>
```

Listing 1: Testfall in PHP

```
ao-suse-ws1.ao-dom.alte-oldenburger.de - PuTTY
ao-suse-ws1:/srv/www/symfony/natural # ./symfony test:unit ComparedNaturalModuleInformation
1..13
# Empty Information
ok 1 - Has no catalog sign
ok 2 - Source has to be created
# Perfect Module
ok 3 - Right modulename selected
ok 4 - Source sign shines global
ok 5 - Catalog sign shines global
ok 6 - Source sign shines at ENTW
ok 7 - Catalog sign shines at ENTW
ok 8 - Source sign shines at QS
ok 9 - Catalog sign shines at QS
ok 10 - Source sign shines at PROD
ok 11 - Catalog sign shines at PROD
ok 12 - Source sign shines at SVNENTW
ok 13 - Catalog sign is empty at SVNENTW
# Looks like everything went fine.
ao-suse-ws1:/srv/www/symfony/natural #
```

Abbildung 7: Aufruf des Testfalls auf der Konsole

A.8 Benutzerdokumentation

Ausschnitt aus der Benutzerdokumentation:

Symbol	Bedeutung global	Bedeutung einzeln
	Alle Module weisen den gleichen Stand auf.	Das Modul ist auf dem gleichen Stand wie das Modul auf der vorherigen Umgebung.
	Es existieren keine Module (fachlich nicht möglich).	Weder auf der aktuellen noch auf der vorherigen Umgebung sind Module angelegt. Es kann also auch nichts übertragen werden.
	Ein Modul muss durch das Übertragen von der vorherigen Umgebung erstellt werden.	Das Modul der vorherigen Umgebung kann übertragen werden, auf dieser Umgebung ist noch kein Modul vorhanden.
	Auf einer vorherigen Umgebung gibt es ein Modul, welches übertragen werden kann, um das nächste zu aktualisieren.	Das Modul der vorherigen Umgebung kann übertragen werden um dieses zu aktualisieren.
	Ein Modul auf einer Umgebung wurde entgegen des Entwicklungsprozesses gespeichert.	Das aktuelle Modul ist neuer als das Modul auf der vorherigen Umgebung oder die vorherige Umgebung wurde übersprungen.